

COMPTE RENDU

Champion des Maths
Mini-projet — Groupe A — Langage C

Étudiant : Enzo Gidaszewski

Module : Langage C

Date : mars 2026

Établissement : INSA / UPHF Hauts-de-France

Sommaire

Introduction.....	2
II.1 — Menu principal et navigation.....	2
II. 2 — Mini-jeu Addition.....	3
II. 3 — Mini-jeu Soustraction.....	3
II.4 — Mini-jeu Multiplication.....	4
II.5 — Tables de multiplication.....	4
II.6 — Mini-jeu Division	5
III. Difficultés rencontrées et solutions apportées	6
IV. Commentaires et suggestions	7
V. Auto-évaluation	7
VI. Lien GitHub.....	8

Introduction

Ce projet a été réalisé dans le cadre du module de Langage C en première année de licence Mathématiques à l'INSA / Université Polytechnique Hauts-de-France. L'objectif était de concevoir un programme interactif en C intitulé Champion des Maths, permettant à un utilisateur de s'exercer sur des notions mathématiques de niveau CM1 : addition, soustraction, multiplication, tables de multiplication et division.

II.1 — Menu principal et navigation

Au démarrage, le programme affiche un menu encadré listant les 5 mini-jeux et l'option de sortie. La variable score est initialisée à 0 en début de programme et s'affiche à chaque tour de menu. La boucle `do...while(choix != 0)` fait tourner le jeu jusqu'à ce que le joueur entre 0. Le `switch/case` redirige vers le bon mini-jeu.

```
srand(time(NULL));

int choix;
int a, b, resultat, reponse;
int score = 0;

do
{
    printf("\n+-----+");
    printf("|1 : Addition           |\n");
    printf("|2 : Soustraction       |\n");
    printf("|3 : Multiplication      |\n");
    printf("|4 : Tables des multiplications |\n");
    printf("|5 : Divisions           |\n");
    printf("|0 : Sortir du jeu       |\n");
    printf("+-----+");
    printf("Score actuel : %d point(s)\n", score);
    printf("Quel est votre choix ? ");
    scanf("%d", &choix);
}
```

Le score est affiché deux fois : dans le menu à chaque tour pour que le joueur voit sa progression, et une dernière fois au message d'au revoir quand il quitte. Le default du `switch` gère tout choix invalide (ex : 9, -1).

II. 2 — Mini-jeu Addition

Deux entiers aléatoires entre 0 et 100 sont générés avec `rand() % 101`. L'utilisateur doit trouver leur somme. En cas de bonne réponse, le score augmente de 1 grâce à `score++`.

```
case 1:
    printf("\nAddition\n");
    a = rand() % 101;
    b = rand() % 101;
    resultat = a + b;
    printf("%d + %d = ?\n", a, b);
    printf("Entrez le résultat : ");
    scanf("%d", &reponse);
    if (reponse == resultat)
    {
        printf("Bravo ! +1 point\n");
        score++;
    }
    else
        printf("Perdu ! La réponse était %d\n", resultat);
    break;
```

`rand() % 101` donne un reste de division entre 0 et 100 inclus. Le générateur est initialisé une seule fois au tout début avec `srand(time(NULL))` pour avoir des nombres différents à chaque lancement. Sans cela, la séquence serait toujours identique

II. 3 — Mini-jeu Soustraction

Le résultat doit toujours être positif ou nul. Deux nombres entre 0 et 100 sont générés, puis échangés si le premier est plus petit que le second. Cette technique s'appelle la permutation par variable temporaire.

```
printf("\nSoustraction\n");
a = rand() % 101;
b = rand() % 101;
if (a < b)
{
    int temp = a;
    a = b;
    b = temp;
}
if (reponse == resultat)
{
    printf("Bravo ! +1 point\n");
    score++;
}
else
    printf("Perdu ! La réponse était %d\n", resultat);
break;
```

La variable `temp` est indispensable : sans elle, écrire `a = b` écraserait la valeur originale de `a` avant de pouvoir la copier dans `b`.

II.4 — Mini-jeu Multiplication

Les deux opérandes sont des entiers entre 1 et 10 (`rand() % 10 + 1`), conformément au sujet. Le principe est identique à l'addition avec le même système de points.

`rand() % 10` donne 0 à 9. En ajoutant +1, on obtient 1 à 10. On évite le 0 car multiplier par 0 est trivial.

```
printf("\nMultiplication\n");
a = rand() % 10 + 1;
b = rand() % 10 + 1;
resultat = a * b;
printf("%d x %d = ?\n", a, b);
printf("Entrez le résultat : ");
scanf("%d", &reponse);
if (reponse == resultat)
{
    printf("Bravo ! +1 point\n");
    score++;
}
```

II.5 — Tables de multiplication

L'utilisateur choisit une table entre 1 et 10. Le programme valide ce choix, affiche la table complète, puis propose 10 questions dans l'ordre croissant. Chaque bonne réponse rapporte 1 point, ce qui peut donner jusqu'à 10 points pour ce mini-jeu.

```
if (table < 1 || table > 10)
{
    printf("Table invalide.\n");
    break;
}
printf("\n--- Table de %d ---\n", table);
for (int i = 1; i <= 10; i++)
    printf("%d x %d = %d\n", table, i, table * i);
printf("\nA vous de jouer !\n");
for (int i = 1; i <= 10; i++)
{
    printf("%d x %d = ? ", table, i);
    scanf("%d", &reponse);
    if (reponse == table * i)
    {
        printf("Gagné ! +1 point\n");
        score++;
    }
    else
        printf("Perdu ! La réponse était %d\n", table * i);
}
```

Le premier for affiche la table (phase d'apprentissage), le second interroge le joueur (phase d'exercice). La validation `table < 1 || table > 10` utilise l'opérateur logique OU (`||`) : si l'une des deux conditions est vraie, la table est rejetée et le `break` sort du `switch`.

II.6 — Mini-jeu Division

Pour garantir une division exacte (sans virgule), on génère d'abord le diviseur `b` et le quotient

souhaité, puis on calcule `a = b × résultat`. Ainsi `a / b` tombe toujours sur un entier. La bonne réponse rapporte 1 point.

```
b          = rand() % 10 + 1;
resultat = rand() % 10 + 1;
a          = b * resultat;
printf("%d / %d = ?\n", a, b);
printf("Entrez le résultat : ");
scanf("%d", &reponse);
if (reponse == resultat)
{
    printf("Bravo ! +1 point\n");
    score++;
}
```

Si on avait généré `a` et `b` directement, `a / b` pourrait ne pas être entier (ex : `7 / 3 = 2.33...`). En construisant `a` à partir de `b` et du résultat voulu, on garantit toujours une division exacte.

III. Difficultés rencontrées et solutions apportées

1. Nombres aléatoires identiques à chaque lancement

Sans `srand(time(NULL))`, `rand()` retourne toujours la même séquence de nombres à chaque exécution du programme. La solution a été d'appeler cette initialisation une seule fois au tout début du `main()`, avant toute utilisation de `rand()`.

2. Résultat négatif pour la soustraction

`rand()` peut générer $a < b$, ce qui donnerait un résultat négatif, non souhaité pour un jeu niveau CM1. La solution est la permutation classique par variable temporaire : `if (a < b) { temp = a; a = b; b = temp; }`. Après cela, a est toujours $\geq b$, donc $a - b \geq 0$.

3. Division non entière

Si on génère a et b aléatoirement, a / b peut ne pas être un entier (ex : $13 / 4 = 3.25$). La solution a été d'inverser la logique : générer d'abord le diviseur b et le résultat voulu, puis calculer $a = b * \text{Résultat}$. On est ainsi certain que a / b est toujours un entier exact.

4. Gestion du score partagé entre tous les mini-jeux

Le score doit être une seule variable persistante pendant toute la session. La solution a été de déclarer `int score = 0` au début du `main()`, avant la boucle `do...while`. Comme tous les mini-jeux sont dans le même `switch` à l'intérieur de cette boucle, ils ont tous accès à la variable `score` et peuvent la modifier directement avec `score++`.

5. Comprendre le rôle du `break` dans le `switch`

Au début, il n'était pas évident pourquoi chaque case se terminait par `break`. Sans `break`, l'exécution continue dans le case suivant, ce qui exécutera plusieurs mini-jeux à la suite sans vouloir. La solution a été de systématiquement terminer chaque case par `break`.

IV. Commentaires et suggestions

Remarques générales

Ce projet était une bonne introduction concrète à la programmation en C. La progression par étapes imposée par le sujet (menu d'abord, puis un mini-jeu à la fois) a permis de construire le programme sans être submergé. Chaque étape était testable indépendamment, ce qui facilitait la détection des erreurs.

Suggestions d'amélioration

Il serait intéressant d'ajouter une difficulté progressive : commencer avec de petits nombres, puis augmenter la plage au fil des bonnes réponses. On pourrait aussi proposer un mode chronomètre pour rendre le jeu plus difficile, pour ajouter un peu de pression sur l'utilisateur.

Apprentissages réalisés

Sur le plan technique, ce projet a permis de maîtriser : la structure `do...while`, le `switch/case`, l'utilisation de `rand()/srand()`, les pointeurs de base, et la lecture/écriture dans des fichiers texte avec `fopen/fprintf/fscanf`. Sur le plan méthodologique, travailler étape par étape (comme demandé dans le sujet) s'est révélé très efficace pour éviter de se perdre dans le code.

V. Auto-évaluation

Critère	Note /10	Commentaire
Fonctionnalités implémentées	8.5/10	5 mini-jeux + score fonctionnels
Qualité du code (lisibilité, commentaires)	8/10	Code clair et structuré
Compréhension des concepts C	7/10	boucles et switch bien réalisés
Respect des consignes	9/10	Structure du projet suivi, étapes respectées dans l'ordre

Pour les prochains projets, l'objectif principal est de mieux maîtriser les pointeurs et la gestion de la mémoire en C, ainsi que de prendre l'habitude de tester le code au fur et à mesure plutôt qu'en une seule fois à la fin.

VI. Lien GitHub

<https://github.com/gidasenzo-dev/champion-maths>